LAB SHEET 5.2

Custom Topologies with the Mininet Python API

Week	5
Duration	2 hours
Deliverable	custom_topo.py, screenshot report (PDF), pushed to your netprog-2026 GitHub repository

Learning Objectives

By the end of this lab, you should be able to:

1. Explain the core classes of the Mininet Python API: Topo, Mininet, and Link.
2. Write a custom topology by subclassing Topo and using addHost(), addSwitch(), and addLink().
3. Instantiate and run a custom topology from a Python script using Mininet(), net.start(), net.pingAll(), and net.stop().
4. Configure per-link parameters (bandwidth, delay, packet loss) using TCLink.
5. Verify connectivity and measure the effect of configured link parameters using ping and iperf.

Background

Lab 4 used mn's built-in --topo generators (single, linear, tree), which can only produce symmetric, pre-defined shapes. Today you move to the Mininet Python API, which lets you describe any arrangement of hosts, switches, and links you can imagine — including asymmetric shapes no built-in generator can produce, and per-link performance characteristics that make individual links behave like slow WAN links, lossy wireless hops, or fast LAN segments, all within the same topology.

Topo (subclass)	You subclass mininet.topo.Topo and override build(); inside it, self.addHost(name), self.addSwitch(name), and self.addLink(node1, node2, **params) describe your topology's shape
Mininet(topo=..., link=...)	Builds an actual running network from a Topo instance. Passing link=TCLink is required for any bw / delay / loss parameters given to addLink() to actually take effect
net.start() / net.stop()	Brings the emulated network up and tears it back down — always pair these, even if your script crashes partway (see the Troubleshooting section).

net.pingAll()	The scripted equivalent of the CLI's pingall — call it directly from Python instead of dropping into the interactive prompt.
CLI(net)	Drops you into the familiar interactive mininet> prompt from Lab 4, using the network your script just built
TCLink is not automatic addLink(h1, s1, bw=10, delay="20ms", loss=5) silently ignores bw, delay, and loss unless the Mininet() constructor was told link=TCLink. Without it, Mininet builds an ordinary, unconstrained veth link and your parameters simply do nothing — with no error or warning.	

Prerequisites

- Lab 4 completed (comfort with the mininet> CLI: nodes, net, pingall, iperf).
- Comfort with basic Python classes and methods (from Labs 2-3).

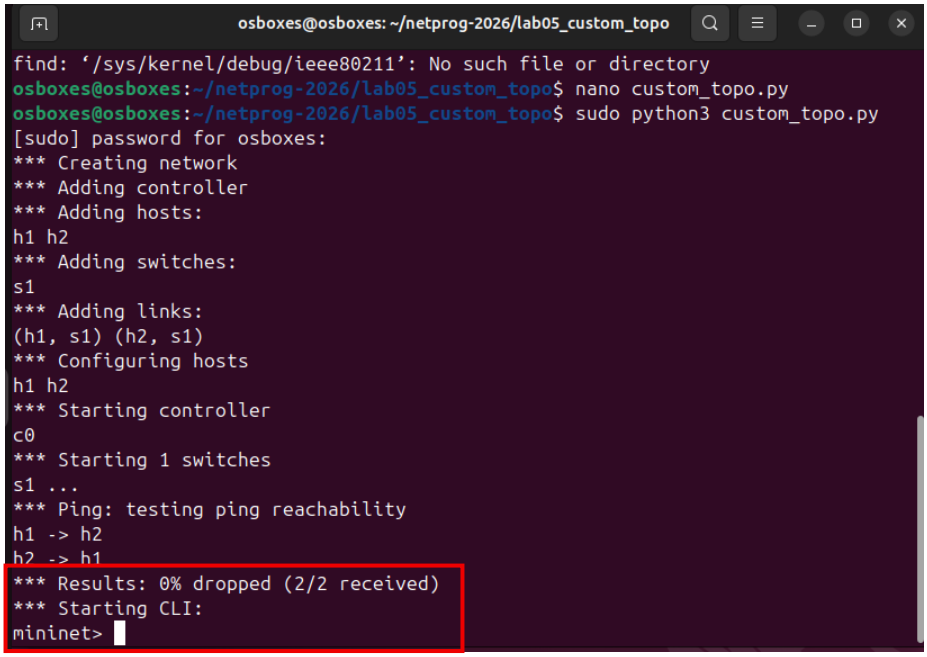
Software & Materials

- Mininet (installed in Lab 1), Python 3.8+
- A text editor / IDE
- iperf/iperf3

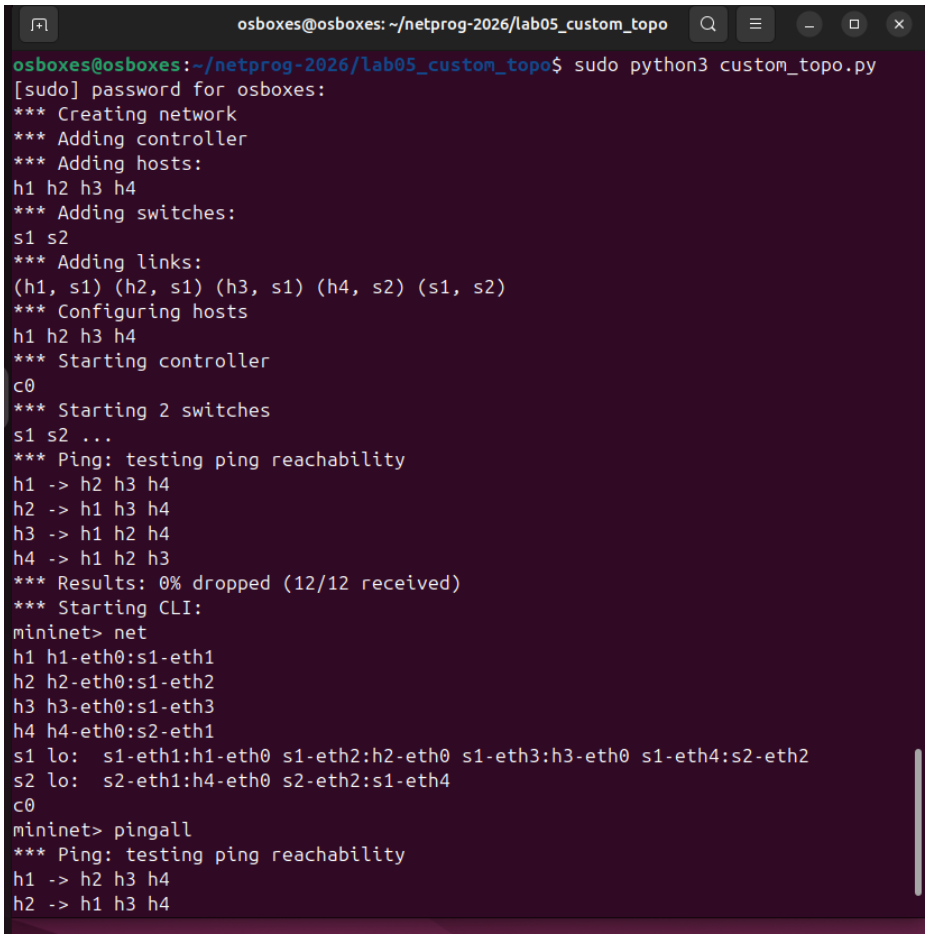
Lab Tasks

Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

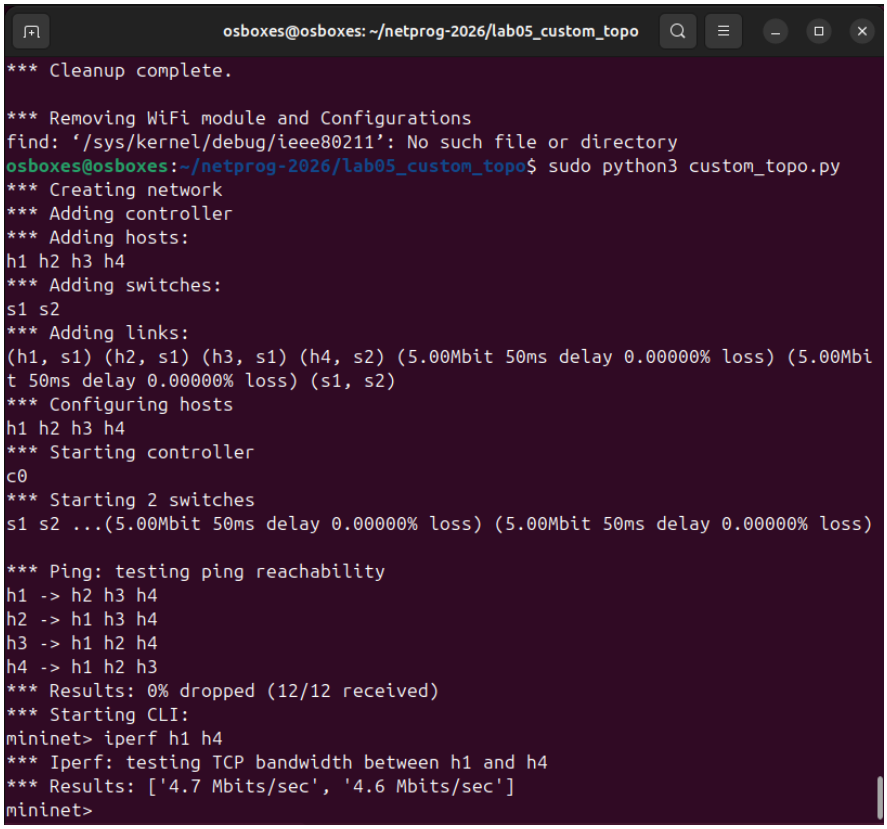
1. **Task 1: Screenshot of Part B's minimal topology with net.pingAll() reporting 0% dropped.**



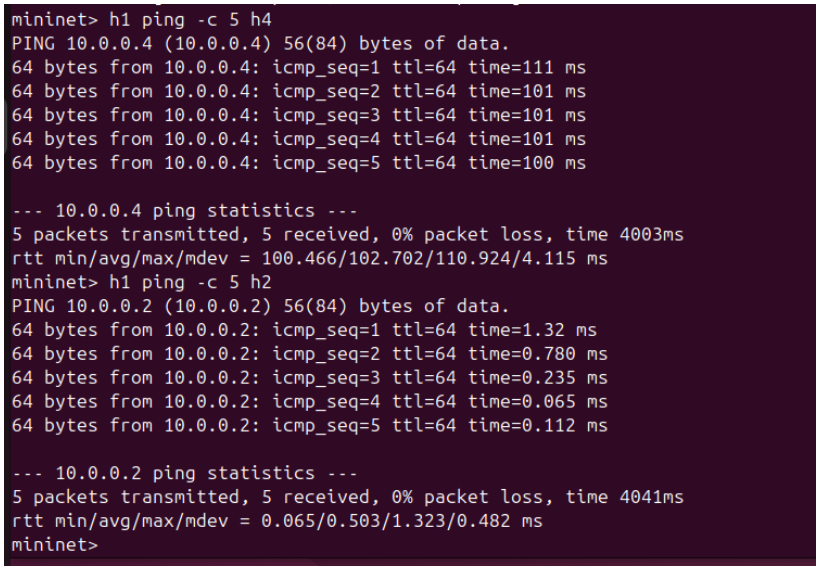
2. Task 2: Screenshot of Part D's net output for CustomTopo, showing the asymmetric 3-hosts-on-s1 / 1-hoston-s2 shape.



3. Task 3: Screenshot of the iperf h1 h4 result from Part F, showing the measured bandwidth is capped near the configured 5 Mbit/s (not the multi-Gbit/s figure you saw on unconfigured links in Lab 4).

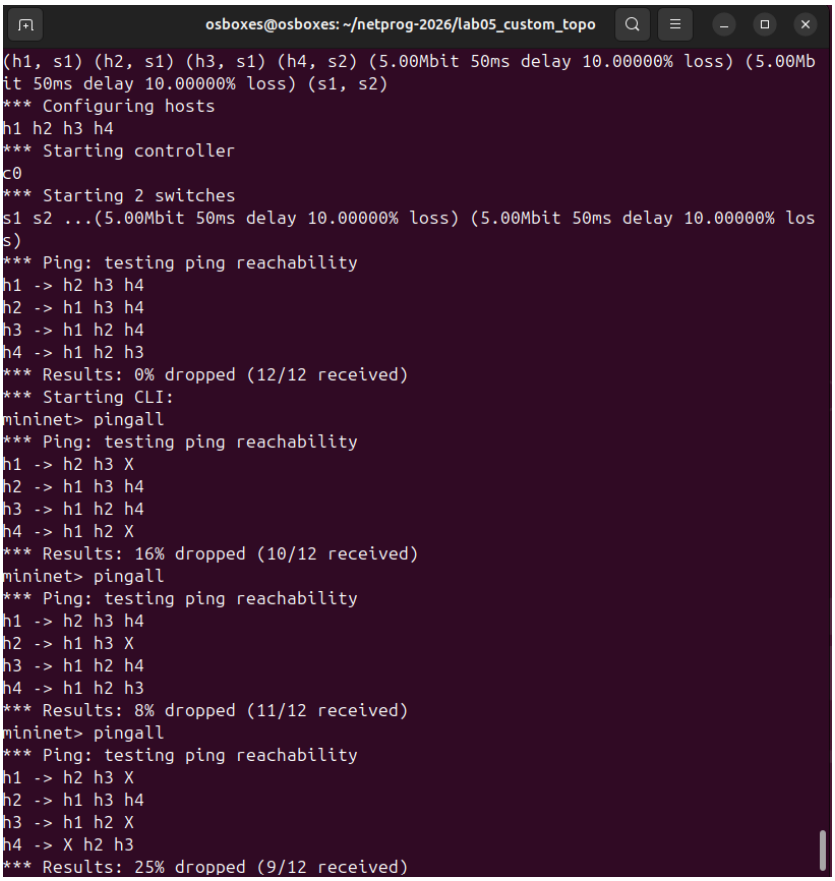


4. Task 4: Screenshots of both ping tests from Part F (h1-h4 across the backbone vs. h1-h2 on the same switch). Report both average RTTs and explain, using the delay value you configured, why the h1-h4 figure is roughly what it is.



Answer I set the delay to 50 ms on the backbone link. A ping crosses that link twice, once going and once coming back. So, the RTT is about 50 + 50 = 100 ms. The h1-h2 ping stays on one switch, so it has no backbone delay. Put your own measured averages in the report.

5. **Task 5: Screenshot of at least one pingall run from Part F showing a nonzero (but not 100%) drop rate caused by the backbone link's loss value you configured. If you see 0% drop every time, run pingall a few more times and report what you observe.**



Answer I set loss=10 on the backbone link, so each packet crossing it has a 10% chance of being dropped. A ping needs a request and a reply, so both cross the link. That gives roughly 19% loss for pings between h4 and the other hosts. Only pings that use the backbone are dropped.

6. **Task 6: In 2-3 sentences, explain why describing topologies in Python (as you did today) is more powerful than Lab 4's CLI --topo generators, referencing something concrete you built today that no built-in generator could have produced.**

Anwer Python lets me build any shape I want. Today I built a network with 3 hosts on s1 and 1 host on s2. The single, linear and tree generators can't make that. Python also let me give one link its own speed, delay and loss while the other links stayed fast.

9. Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. **Explain the role of the build() method in a Topo subclass — when is it called, and by what?**

   Answer Role of build() is where you add the hosts, switches and links. Mininet calls it automatically when you write Mininet(topo=MinimalTopo()). You never call it yourself.

2. **Why must link=TCLink be passed to the Mininet() constructor for bw, delay, and loss parameters on individual links to actually take effect?**

   Answer The default link is a plain link and ignores bw, delay and loss. TCLink uses Linux traffic control (tc) to apply those settings. Without it, Mininet gives no error and the settings do nothing.

3. **Describe a real network scenario you could model with a custom topology and per-link parameters that neither of Lab 4's built-in generators (single, linear, tree) could represent.**

   Answer A branch office with 3 computers on a fast office switch. That switch connects over a slow, 100 ms WAN link to a data center with 1 server. The built-in generators can't make this shape, and they can't give one link different speed, delay and loss.

End of Lab Sheet 5.2